

ABRA — roadmap

2026-08-01

The bar for the completed list is one thing: **would I defend this sentence for twenty hours against someone trying to break it.** Anything that fails that test is in section 2 or section 3, however much work went into it.

A result is only here if it is (a) measured rather than argued, (b) measured on data that passes `engine/quality.js` , and (c) still true when re-run today. Several things that were on this list a week ago have been removed for failing (c).

1. DONE — and defensible

1.1 The data is trustworthy, and we can prove which parts are not

87% of the ladder store is unusable and we know exactly why. 15,973 stored games, ~2,057 clean. Bots, forfeits, partial brings and stubs, each counted separately in a funnel that anyone can re-run. 27 assertions in `tests/test-quality.js` .

Behavioural bot detection works on accounts that do not announce themselves. Team invariance: an account playing 50+ games without ever changing a slot is a script. It caught six accounts that appeared in **52.2%** of the games a name-only filter called clean. Carchdraw84172 played 459 games with one team, 367 of them in a single day.

Defensible because: the rule is stated, the evidence is per-account, and the false-positive check is published — no account with ≥ 50 games in the store has more than one distinct team, so the rule separates cleanly.

Known limit, stated: a bot playing under 50 games or varying its team is invisible. The right phrase is always "no bot detected", never "human".

1.2 The damage engine is correct

31 of 31 scenarios within 2% of `@smogon/calculator` , through the official Showdown engine pinned at commit `20ad99f` . Golden-master, re-run before every engine change.

Defensible because: it is validated against an independent implementation, not against itself, and the harness caught two of its own wiring bugs on first run.

1.3 MAGNEMITE reads the board, and the improvement is measured out of sample

600 seed-matched battles per policy — same teams, same seeds, same engine — against 1,905 clean ladder games:

	before	after	real humans
super effective	9.71%	14.91%	21.37%
moves that outright failed	9.68%	6.34%	2.47%
moves that hit an immune target	4.30%	2.92%	1.91%

Defensible because: the fit never saw the realism report — it was held back as the out-of-sample check — and the comparison is paired on seeds so team difficulty cancels.

What this does NOT claim: that MAG is good. See 2.1.

1.4 Imitation is a ceiling, and we have the number

A challenger optimised for **winning** rather than for resembling humans, drawn from MAG's own 17 features, beats MAG **63.2%** — and beats MAG's predecessor **68.2%** where MAG itself manages **60.2%**.

Defensible because: the 68.2% is an independent evaluation the search never optimised against, on an opponent it was not tuned for. It is transitively better, not a counter.

RETRACTED 2026-08-04 — all three figures describe a 17-feature vector we no longer ship. They were computed on an engine 25 wire-fixes old and **before the quality filter existed**; provenance.js carried data/exploitability.json as its only UNSAFE artifact on exactly that ground. The re-run on the shipped 58 features is **void** — data/policy-weights.json was refitted at 22:15:24 UTC while the search was running and engine/medicham2-browser.js moved twice more afterwards — and its hill-climb accepted **1 of 24** steps, so it was uninformative regardless. See docs/SEARCH.md §R8. **The claim of this section may still be true; the evidence for it no longer exists.** The general point that imitation and winning are different objectives stands on its own separate evidence (see MAGNEMITE in docs/MODELS.md), not on these three numbers.

And it is interpretable: the winning weights barely care about type effectiveness and care enormously about not wasting a turn and finishing what is already hurt.

1.5 The open-sheet corpus is safe to learn from, and we checked rather than assumed

Teams differ enormously between open- and closed-sheet play — **551.9 points** of total absolute species difference, Garchomp on 81.6% of open-sheet teams against 47.7% of closed. Behaviour given a board differs by **at most 1.49 points** on every measure.

Defensible because: both sides were measured with the same code, and the policy is conditional on the board — it never learns what to bring, and MEW draws teams from the ladder store regardless.

engine/fit_policy.js re-checks it on every refit by re-estimating on a reweighted sample.

1.6 Collection is complete and automatic

The Bo3 **open-team-sheet** ladder is now pulled hourly into its own store. That format carries Force Open Team Sheets , so every game publishes all six sets — the only continuously-collected corpus where the choice set of a decision is known rather than guessed.

Defensible because: the format's ruleset was read from the simulator, the pull was tested live, and the store is separate — Bo3 is a different information regime and pooling it would corrupt every behavioural statistic.

1.7 Facts the model can see are derived, never typed

Ability blocking (Levitate/Ground, Flash Fire/Fire, Good as Gold/status, Armor Tail/priority) is **read out of 14,933 recorded battles**, not written down — including inferring *which rule* each ability follows by testing candidate hypotheses and preferring the narrower on ties.

Defensible because: it was verified against clean-only data and every rule is identical; and because probing Showdown's handlers directly was tried first and **failed silently**, which is why the measured route was taken.

1.8 The project can audit itself

- engine/provenance.js — every artifact checked for staleness against the filter, its inputs, and the clean-game count. Found **28 unsafe to quote**.
- engine/conformance.js — S1–S13 as executable checks over 132 files. **S12 is now clean**.
- engine/selftest.js — 24 assertions on failures that are silent by nature.
- build/build_pdfs.js — every document has a current PDF, list derived.

Defensible because: each found real defects on its first run, including in the tool that found them.

2. NOT DONE — and specifically why

2.1 MAG is not good

It is **exploitable** (1.4), it still repeats Protect at roughly double the human rate, it wastes 6.34% of moves against a human 2.47%, and its one head-to-head win was against a baseline with a flaw it was purpose-built to punish. **It is a starting position, not a player.**

2.2 The self-play ladder has produced no evidence

One confirmed promotion. The round robin that would establish whether the ladder climbs or goes in circles has never had enough games to resolve anything. Reported as a null.

2.3 ~Twenty-eight artifacts are unsafe to quote~ — CLOSED 2026-07-31

o UNSAFE, 45 ok. Twenty-five were regenerated by re-running their own generators; the last was fixed by addressing what the gate was actually complaining about. `provenance.js --strict` is wired into the suite and green. This was named "the blocker on everything in section 3" and it is no longer blocking anything.

Two real bugs were underneath the staleness, neither of which waiving by name would have found: `illusion.json` carried a sound raw-store justification in its generator that never travelled to the artifact; `archetypes.json` reported **team sides** (2 per game) under the name `n_games`, so every share and silhouette in it had the wrong sample size on its label.

What this does NOT establish. `provenance.js` says so itself: it checks what artifacts DECLARE. **o** UNSAFE means nothing is misdeclared, not that every number is right. **14 are still flagged "possibly stale"**, and only re-running a generator can settle whether its output is correct.

2.4 Models that are retracted, null, or without usable input

Recorded so they are not quoted again:

model	state
PORY	its comparison artifact was trained without the filter; standing unknown , not weak
SLOWKING	mathematics sound, but the clean matrix has o decisive matchups — no usable input
GURU	o decisive matchups on clean data; descriptive only
JOLTEON	ties a coin; recommended for retirement
WAR	withdrawn — the signal was four bots playing one team 1,446 times
CHOMP-EV	honest null; brings do not separate from a coin
the 199,524-game self-play corpus	unusable — generated before megas worked, with duplicate Protects on one set
"the meta is rock-paper-scissors"	withdrawn; computed pre-filter, deleted

2.5 Nothing here handles hidden information

Self-play has none — we run both sides. Nothing in the current work teaches the bot to play against an unknown team, which is the defining difficulty of the closed-sheet ladder.

2.6 Closed since this list was written (2026-07-29 → 30)

Items 3, 4 and 5 below have shipped. Recorded rather than deleted, because a roadmap that quietly drops its own entries cannot be audited.

- **"Give MAG a damage calculation, and a KO check both ways"** — DONE. `board.js` calls the real damage engine throughout; `koTarget`, `killIsRoll`, `diesBeforeMoving` and the switch-survival features all

read it.

- **"Score both Pokémon together"** — BUILT AND MEASURED, and it **lost**: 42.0% [39.9, 44.3] over 1,934 paired games, 28.4% of decisive pairs, against its own zeroed control. The imitation fit is refuted; the idea is not. Now item 1 below.
- **"Switching"** — no longer a coin flip. Voluntary switches score through `switchFeatures`, the post-KO replacement is scored rather than rolled, and `entryHits = forced ? 0 : 1` separates the two cases.

A process failure that caused real damage, recorded so it is not repeated. On 2026-07-30 the model family was repeatedly mischaracterised in conversation — DODUO described as unbuilt when it had been built, wired, controlled and measured; MEDICHAM audited in its graveyard version; top-K pruning "proposed" when `fit_joint.js` already had it. Cause: two full days and ~40 commits of work existed in NO document. `MODELS.md` was last written 2026-07-28; every other living doc 2026-07-26. The only reason DODUO's verdict was findable at all is that its experiment happened on the one day `MODELS.md` was touched. **CLAUDE.md requires the docs to move in the same pass as the code. They did not.**

2.7 Closed 2026-07-31

- **"Choice lock in the candidate set"** (was item 2) — DONE. The logit denominator no longer contains moves the human could not click. After the refit, six of eight switch features clear zero, and **switches now win the argmax** where `--switching` previously changed nothing.
 - **The tag artifact reaches the feature vector** — `speedSwing +0.983`, `screenValue +1.128`, `healValue +2.220`, all clearing zero by wide margins. 53 → 56 features. This is the first feature addition since 3.28.0's four nulls to measure as real, and the difference is that these are CONDITIONS on mechanics with no prior representation rather than restatements.
 - **ALAKAZAM job 2 (guess what they will do)** — built, off by default, needs a refit. The max in `incomingThreat` is now an expectation over P(their action).
 - **A gate against wasted runs** — `engine/preflight.js`, 24 games, refuses a training run whose feature blocks receive no gradient. Two 1.5-hour runs were lost to exactly that before it existed.
-

2.8 Closed 2026-07-31 (evening)

- **DODUO is trainable for winning** (was item 1, the top of section 3). Wiring only, as the item predicted. `--joint` trains the 74-length vector on the game result; `--joint-only` freezes the 56 singles so the whole trust-region step reaches the 18 pair terms. Four separate faults would each have produced a plausible, wrong learning curve, and every one was caught by a check rather than by reading. **The head-to-head is running; no win rate is claimed.**
- **The trust region starved the pair block, structurally.** The first run moved it 4.7% while the singles moved 21.7% — single features appear in nearly every choice set, pair features are sparse, so they lose a shared budget by construction and more iterations scale both equally. `--joint-only` took it to **40.0%**, with three sign flips: `overkill -0.951 → +0.980`, `focusFireKills -0.107 → +1.094`, `partnerCoversMe -0.004 → +0.639`, and `bothSameTarget 0.031 → 2.249`. The imitation fit penalised focus fire; training to win reverses it.
- `mew_farm` **exited 0 with every worker dead.** A farm that wrote zero games was indistinguishable, to anything checking the exit code, from a clean run. Found because the DODUO gate refused to start.
- **The speed-multiplier divergence is cross-checked** (systems audit R2, which the audit could not finish because `board.js` was off-limits mid-run). `tests/test-speed-multipliers.js`, 15 passed, compares

behaviour as ratios and compares the SETS, so a new weather-speed ability in the dex cannot hide behind four passing constants.

- **A test that asserted a count.** `sanity_check.py` required Sun > 1000 teams; Sun is the third largest of eight styles. Now relative and mutation-tested. **96 passed, 0 failed.**
- **All twelve recommendations from the three 2026-07-31 reviews are closed.** Including the two that had been built but never reached a reader: the multiplicity correction (on disk since 14:04, in no reader-facing document) and the retired NMF justification, still live in `PUBLICATION.md` — the document that exists to be published — while the honest version sat in `SUMMARY.md`.
- **There is a rollback point.** `v3.31.0`, tagged and pushed, with the one known failure named in the tag message. Both reviews had recorded that nothing existed to roll back to.

2.9 Closed 2026-08-01 (live play against a human)

Will played MAG in the real Showdown client for the first time — six games. **Eleven defects in about two hours**, more than the 47 test files had surfaced. Full evidence: `docs/FINDINGS-2026-08-01-live-play.md`.

Handoff: `docs/HANDOFF-2026-08-01.md`.

- **The bot was not delivering the model to the board.** `makeScoringPlayer(opts)` accepted an options object and never used it — the class reads its own second constructor argument — so `{greedy: true, switching: true}` configured nothing. MAG played its first real games *sampling* its moves instead of taking its best one. Fixed by merging factory options into constructor options.
- `joint: true` **silently disabled mega evolution.** `_withMega` had exactly one call site and the joint path returned around it. Both joint paths now go through it, and `test-wiring.js` re-asserts mega, aiming and open sheets **under** `--joint` — the broken cell was the intersection of two capabilities each tested alone. Fourth mega defect in this project.
- **Voluntary switching was reverted to off.** The bot had requested it since its first version, against `mew.js:135` 's measured verdict of a **10-point loss** — **RETRACTED 2026-08-06 as unattributable and confounded: it ran through medicham2 playouts predating WIRES 123-128, its artifact carries no `engine_release` stamp, and `bringIn()` selects `live(bench)[0]` so the experiment measured switching to an ARBITRARY body, which the engine itself calls "evaluating LEAVE" rather than evaluating a switch. See #63.** It only took effect once the options bug was fixed — an unmeasured change against a measured result.
- **Item Clause repair could never succeed.** It resampled while telling the sampler the answer had to be the colliding item, so the "rare" no-item fallback fired every time.
- Open team sheets are now accepted; failed team draws redraw instead of rejecting the challenge.

Nothing about MAG's playing strength was established. MAG went 1–2 in the three games where the policy was actually driving. Six games is not evidence in either direction.

3. NEXT — in order, with the reason each is next

o. REFIT — and it may rewrite item 1 below. `board.js` changed a feature's *meaning* under an unchanged *name*, so every shipped weight was fitted against the old definition. `board.js` is required by 40 files; the vector is read by 24. Nothing in the project detects this: `magnemite.js:297` compares feature names and `:299` compares vector length, and both pass when semantics drift. A guard — a stored hash of each feature's values over a fixed fixture board — is the cheapest high-leverage item on this list, and the only one that prevents a repeat rather than fixing an instance.

Why this comes before item 1, and may dissolve it. Item 1 rests on `spreadFreeBesideAlly` being priced at -5.054 , read as "humans rarely click this, not this is bad." That feature fires only when `allyHit === 0`, and `allyHit` used `getEffectiveness >= 0`, which **returns 0 for an immunity**. So the feature's coverage was inverted at its most important case:

ally beside a spread move	did <code>spreadFreeBesideAlly</code> fire?	is it actually free?
Flying partner + Earthquake	no	yes — fully immune
Levitate partner + Earthquake	yes (ability path)	yes
merely resisting partner	yes	no — it still takes damage

The clearest free-spread case in VGC never fired once, while a case that costs real HP did. A weight fitted on that is a weight fitted on the wrong question, and **-5.054 may be an artifact rather than a preference**. Stated as a hypothesis; the refit is the test. If it survives the refit, item 1 stands exactly as written.

RESOLVED 2026-08-01 — it WAS an artifact, and of something else entirely **The immunity hypothesis above is refuted, and the conclusion it was reaching for is confirmed by a different cause**. Both fits were re-run. **The refit alone changed almost nothing**. `allyHit` moved from -0.0187 to -0.0126 (unweighted $-0.0937 \rightarrow -0.0875$) — about **0.2 standard errors**, i.e. noise. Measured rather than argued: the immunity fix changes **0.198% of candidates**, and `allyHit` fires on only 0.63% of them to begin with. So the honest reading of `allyHit` is the plain one — **humans do not avoid hitting their own partner much** — and it should be reported that way rather than re-explained. **What produced -4.986 was the FITTER, not the feature**. `fit_joint.js` matched a human's click by requiring the candidate's target to match, and `board.js` builds a spread candidate with `targetMon: null` because Earthquake is not aimed. **No spread click could ever match**. Measured over 400 corpus games: spread moves are **14.94% of all human move clicks**, 99.71% carry a recorded target, and **1,393 of 1,397 failed to match**. Because a joint turn needs both slots, the loss compounded — the fit discarded **57,486 of 82,483 joint turns** and estimated 74 weights from 30% of the data. The discarded 70% was not random: it was exactly the turns containing a spread move. `spreadFreeBesideAlly` therefore fired on 8.0% of the enumerated alternatives and **0 of 1,461 pairs a human actually chose**, and a conditional logit reads "always available, never chosen" as a large negative. `fit_policy.js:432` had this right all along, so the **56-feature vector was never affected**. Corrected, on 63,305 usable turns, **nine of eighteen pair weights flip sign**: | pair term | old | refit + matcher fix | |---|---|---| | `spreadFreeBesideAlly` | -4.986 | **+0.863** | | `terrainSetupHelpsPartner` | -4.125 | **+2.005** | | `screenWhileThreatened` | -2.982 | **+0.110** | | `speedSetupHelpsPartner` | -1.595 | **+0.540** | | `weatherSetupHelpsPartner` | -0.549 | **+1.292** | Every "one slot sets up, the partner benefits" term was negative and is now positive. Held out, the joint layer takes top-1 from 9.7% to 11.5%. **And it wins**. Refit+fix against the shipped vector, seed-paired, greedy and joint on both arms, on three disjoint seed blocks: | run | games | decisive pairs to the refit | SPRT decided at | |---|---|---|---| | first | 17,350 | 66.7% | 78 decisive pairs | | replication | 1,934 | 65.9% | 69 | | challenger in arm 1, as convention | 1,932 | **70.9%** [66.0, 75.4] | 71 | The third is the one to quote: `sprt.js` and `paired_h2h.js` were run over it independently and agree to the decimal. All three are far past the 55% ship threshold, and every one was decided inside ~420 games — the fixed 200,000-game run this started as was 99.8% waste. **Item 1's stated premise is void**. It reads -5.054 as evidence that the imitation objective prices good play badly. That number was a fitter defect. Item 1 may still be worth doing — the objective argument has independent support from greedy — but **this is no longer evidence for it**, and DODUO's 42.0% was measured with the contaminated vector and no longer describes the current one. **A correction about the analysers, recorded because it was nearly shipped as a fix**. The first reading of this H2H looked inverted, and `sprt.js` was changed to treat arm 2 as the challenger on the strength of `mew.js:215` calling `--weights2` "the challenger". That comment is about the EXPLOITABILITY search, not the

standard A/B. **Arm 1 is the challenger** — `paired_h2h.js:183` builds its NEW label from arm 1, and the run that measured greedy at 79.7% put `--greedy` on arm 1. The change was reverted; making it would have put the two analysers at odds and inverted every run already analysed. What WAS wrong is narrower: `sprt`'s name-based fallback hardcoded `'score'` as the new arm, so on `--policy prior --policy2 score` it disagreed with its own arm-based path. Fixed, and `tests/test-sprt-arm-sign.js` now asserts the two analysers use the same rule so they cannot drift.

1. DODUO trained for WINNING, not for resemblance. The most precise open question in the project. DODUO is fully built: `fit_joint.js` fits the pair block, `magnemite.js` plays it (`--joint`), `--joint-zero` is a true control running the whole pair path with only the coordination weights zeroed, top-K is capped, and fallbacks are counted in `stats.jointFallback`. It lost at 42.0%, and the fit itself says why — it prices `spreadFreeBesideAlly` at **-5.054**, which means humans rarely click this, not this is bad. A bot told to avoid a free spread move beside its own ally by -5 will decline its best plays.

2026-07-30 established the pattern that makes this item one: **four knowledge additions produced four nulls; two changes to the OBJECTIVE produced two large wins** (greedy +12 points / 79.7% of decisive pairs; self-play 55.9%). The objective is the binding constraint, and DODUO has only ever been fitted to the losing one.

RECONCILED 2026-07-31. That 55.9% was measured on the **53-feature vector with switching OFF**. Repeating the experiment on the **56-feature vector with switching ON** gives **48.1%** [46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (which used to be supported by the older 10-point switching loss — **that figure is RETRACTED 2026-08-06 as unattributable and confounded**: `medicham2` playouts predating WIRES 123-128, no `engine_release` stamp, and `bringIn()` selects `live(bench)[0]`, so it measured switching to an ARBITRARY body rather than to a chosen one. The candidate cause stands; its supporting evidence does not. See #63), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.

THE WIRING IS DONE, 2026-07-31 (commit `ce5367c`). It was wiring, as this item predicted, and the arithmetic in the sentence below was wrong in its parts while right in its total: the vector is **56 singles + 18 pair terms = 74**, not 53 + 21. The pair softmax is $P(q) = \exp(s_q) / \sum \exp(s_k)$ with $s_q = wS \cdot x_a + wS \cdot x_b + wJ \cdot jf$, so the feature vector attached to pair q is the concatenation $[x_a + x_b, jf]$ — the two single vectors **summed**, because both are scored by the same single block. `accumulateLogitGrad` was already generic over length, so no new mathematics was needed. **Four separate faults would each have produced a plausible and wrong learning curve**, and every one was caught by a check rather than by reading the code: | fault | what it would have looked like | |---|---| | `mew.js` sized the run gradient from the **single** weight file and summed with `k < GRAD.length` | 74 truncated to 56 — every pair weight silently dropped, curve printed as normal | | `magnemite.js` allocated `learnGrad` ~50 lines **before** `this.wj` loaded, so `this.joint` was still false | every joint player returns 56 entries; caught by the above on the first 6-game probe | | `magnemite.js` read the joint vector from a **hardcoded** path | every iteration replays the frozen shipped fit; pair terms never move; flat curve reads as convergence | | `preflight.js` indexed only `B.FEATURES` | the gate certifies a joint run **without checking the block that run exists to train** | `--joint` on one arm is the A/B experiment, not a training configuration. `mew.js` now refuses `--learn` with the layer on one side (the run gradient sums both sides, and index k is a different feature in each); `train_policy.js` adds the mirror flag rather than leaving it to be remembered. **Measured:** `preflight` at 24 games with `--joint --joint2 --switching --switching2` — every block live, pair block 15 of 18 live, `|gradient|` 9.12. Two real iterations at 40 games each move the

pair terms on their own merit: `bothSameTarget` **+0.164** (third-largest change in the whole 74-vector), `overkill` **+0.120**, `focusFireKills` **+0.094**. Suite 40 passed / 2 failed, the same two known failures.

TRAINED 2026-07-31 (evening). 12 iterations x 5,000 games with `--joint-only`, which freezes the 56 singles so the whole step reaches the pair block (the first run, unfrozen, moved it 4.7% -- see 2.8). Pair block moved **40.0%** of its own length; singles moved **0.000**, verified, so the head-to-head differs in the 18 coordination weights and nothing else. | pair term | resemble | win | | ---|---|---|---| | `bothSameTarget` | 0.031 | **2.249** | | `overkill` | -0.951 | **+0.980** | SIGN FLIP | | `focusFireKills` | -0.107 | **+1.094** | SIGN FLIP | | `partnerCoversMe` | -0.004 | **+0.639** | SIGN FLIP | | `spreadFreeBesideAlly` | -4.986 | -4.815 | barely moved | Three of the four largest changes are sign flips and they tell one story: **the imitation fit penalised focus fire, and training to win reverses it**. Humans spread damage; winning concentrates it. `spreadFreeBesideAlly` **is the exception and it is this item own headline weight**. The argument above is that a bot told to avoid a free spread move beside its ally by -5 will decline its best plays. After training it is -4.8. Whatever moved the other three did not move this one, and that is unexplained. **WHY IT DID NOT MOVE — ANSWERED 2026-07-31 (Will)**. Not cancellation and not a dead feature: the gradient was **positive in all 12 iterations, never once negative**, so the evidence is unanimous that -4.99 is too negative. It is ~13x smaller per round than `bothSameTarget` because the feature UNDER-FIRES, and the reason is a detector that is too narrow: ``js const freeSpread = (c, x) => c && c.move && c.spread && c.spread.length > 1 && F(x, 'allyHit') === 0; ` allyHit is a SINGLE-SLOT feature computed from the partner's TYPE and ABILITY — Levitate, Flash Fire, Volt Absorb, Earth Eater, or resisting the type. **A partner who clicks PROTECT is spared exactly as completely as a Flying partner, and the feature cannot see it**, because Protect is not a property of the partner; it is a choice the partner makes that turn. That is structural, and it is the whole argument for the pair layer: `spreadFreeBesideAlly` RECEIVES both candidates and reads the `allyHit` of its own slot only. The partner's Protect is already in hand and is never consulted. **Measured cost:** Earthquake is on **69 of 308 species (22.4%)**, by far the commonest spread move, and it is `allAdjacent` — so today it qualifies only when the partner is immune. 40 species (13.0%) carry an `allAdjacentFoes` move that spares the ally automatically; 83 (26.9%) carry an `allAdjacent` move that does not. The commonest real line in VGC — Earthquake plus partner Protect — is in the second group and is invisible. **FIX, QUEUED (board.js was in use by the head-to-head, per the no-edit-during-a-run rule):** extend `freeSpread` to treat the partner candidate as sparing the ally when it is a protect-family move. `PROTECTMOVES` and `tgtMayProtect` already exist, so nothing is typed in. **WIDE GUARD: RESOLVED, and it is BOTH cases depending on whose side it is on. **On MY side it spares my partner from my own spread move exactly as Protect does. On the FOE side it blocks my spread move outright, which is an anti-synergy and belongs in the dead-move tests, not here. **THE FULL SET IS DERIVED, NOT TYPED** — probe each candidate move's volatile/side condition by calling its real `onTryHit` against the actual spread move, the same technique `entryEffects` and `speedStub` already use. Measured 2026-07-31: | spares the ally (10) | excluded, and why | ---|---| | Protect, Detect, Spiky Shield, Baneful Bunker, Burning Bulwark, Silk Trap, King's Shield, Obstruct, Wide Guard, Mat Block | **Quick Guard** — priority only; **Crafty Shield** — status only; **Endure** — no `onTryHit` at all, it prevents FAINTING, not damage | Three flag-based shortcuts were tried and are all WRONG, recorded so they are not retried: `stallingMove` admits **Endure**; the presence of `onTryHit` admits **Quick Guard and `Crafty Shield`; `target === 'allySide'` admits both of those and misses every self-targeting Protect. Only running the handler against the move separates them. **This weakens the interpretation of the head-to-head now running. Both arms share the same detector, so the comparison is still fair, but neither arm can express the coordination pattern the feature is named for. **MEASURED 2026-07-31. IT LOST.** 194,514 paired games, 97,257 complete pairs. | | | ---|---| | decisive pairs | ****26,405**** (27.1% of all pairs) | | NEW (trained to win) takes | ****45.7%****, 95% CI ****[45.1, 46.3]**** | | verdict | ****worse**, and the interval clears

50** | Training the 18 coordination weights on the game result made the policy WORSE than the same weights fitted to imitate a human click. The three sign flips that looked like a coherent story about focus fire do not pay. **The likely reason is in the same table: 72.9% of pairs were 1-1 splits**, i.e. the TEAM decided the game and not the policy. That contamination is in the TRAINING gradient too, so REINFORCE on win/loss was mostly fitting a team lottery. The objective was right; the ESTIMATOR of it was not. Three fixes follow, and none of them is "go back to imitation": 1. **Pair the TRAINING runs** as the tests already are, so team luck cancels out of the gradient. 2. **Score decisions against a win-probability baseline** (PORY) rather than the final result, so a move is credited with what it changed instead of what happened six turns later. 3. **Train against a pool, not only against self** – a self-play policy beats its training partner, and this one had never faced the imitation policy until the head-to-head it lost. That is VGC-Bench's own finding, already recorded in docs/COMPETITORS.md. **CAVEAT, AND IT IS LARGE.** The whole-repo review found `engine/fit_policy.js:170` computes the reweighting factors from ONE store and applies them across all three, so the shipped weight vector was fitted against the wrong population. Both arms here share those singles, so this comparison is internally fair – but every ABSOLUTE number in this project is provisional until that refit. **`engine/sprt.js` reached this verdict after 3,516 games instead of 194,514 – 98.2% of the run saved.** `--verify` confirms it agrees with `paired_h2h.js` exactly (12,073 and 14,332 on both readers). A 136-minute run becomes about two and a half minutes. What remains: ``node engine/train_policy.js --joint --switching --switching2 --procs 6 --iters <N> --games <G>`` then the head-to-head the trainer now prints, in which **both arms run the pair path with identical singles and only the 18 pair weights differ** – so it measures **trained to win** against **fitted to resemble**, and nothing else. The previous printout passed a 74-length file to `--weights`, which *would have been rejected outright or, worse, run an experiment whose arms differed in the wrong block.*

A trap already paid for once: `fit_joint` fits its single block and its pair block TOGETHER, and 23 of 48 features carry opposite signs between that fit and the shipped one. Mixing the two vectors lost 31.2% on decisive pairs and would have been reported as "coordination does not help."

2. Re-run WOBBUFFET against the current vector. (Will, 2026-07-30: put this back on the list.) **ATTEMPTED 2026-08-04 AND VOID – still open, and now there is no number at all rather than a stale one.** ~Its result is still the most important number in the repo and it is three feature-generations stale: a counter found in forty minutes beat MAG **63.2%** [56.6, 69.3] on the 17-feature vector, mirror control 47.5%.~~ The re-run's defender was refitted mid-run and the simulator moved twice more; separately its hill-climb accepted 1 of 24 steps, so `exploit.js` needs a stamp and a dimension-aware step rule before the next attempt. `docs/SEARCH.md` §R8 has the prepared command and its three preconditions. That challenger was not rock-paper-scissors – it was a better player drawn from the same features and optimised for wins rather than resemblance, which is item 1's lesson in another form. `engine/exploit.js --target <weights.json>` can now be pointed at DODUO, so the EXPLOITABILITY argument for coordination – the one the 42.0% result explicitly does not settle – can finally be tested instead of argued.

3. MACHAMP, re-run on the current vector. (Will, 2026-07-30: keep it.) Half-run and stale: the 2026-07-26 run completed 2 of 6 generations on a **17-feature** vector and recorded no verdict; the vector is now 53. Its METHOD is alive – champion/challenger promotion behind a Wilson interval is the same win-objective idea `train_policy.js` now implements by policy gradient. Keep the guard that made it honest: every promoted champion plays EVERY previous generation, so that "gen 5 beats gen 4" is not read as progress by itself. Kept as insurance rather than as evidence – the "this metagame is cyclic" justification is withdrawn (2026-08-02); see `docs/MODELS.md` under MACHAMP.

4. Branch scoring with a mixed strategy — the research is done, so this is no longer vague. Measured on a real mid-game board: $9 \times 8 = 72$ **joint actions per side**, $72 \times 72 = 5,184$ **matrix cells**. The 10^{12} figure from VGC-Bench is the game TREE, not one turn.

- Only **28 of 72** pairs have a non-zero joint vector. The other **44 are free** — their score is exactly the sum of two singles already computed.
- With two Pokémon the coordination graph is a SINGLE EDGE, so Variable Elimination reduces to enumerating the 72 pairs and Max-Plus is unnecessary. The MARL factorisation literature (QMIX, QPLEX, Weighted QMIX) exists to avoid enumeration at many agents, which is not our problem.
- What that literature DOES tell us: QMIX's monotonicity constraint cannot represent non-monotonic coordination — exactly "Protect while my partner removes the threat." Independent per-slot scoring cannot express it either. Not "scores it badly": cannot express it.
- Pruning order matters. Structural prunes (choice lock, spread moves with no target, immunities) are exact and happen BEFORE scoring. Score-based top-K is circular for the 72 and only pays for the 5,184.
- Pokémon is SIMULTANEOUS-move, so the answer at a node is a mixed strategy, not an argmax. Minimax is the wrong algorithm; this is poker-shaped. SLOWKING's regret matching already does this mathematics at team preview and nobody has pointed it at a turn.

5. A leaf evaluator, so depth > 1 is affordable. `position_features.js` (16 features) plus PORY retrained on clean data. Without it every node rolls to the end; with it, depth 2 is 625 cheap evaluations.

6. Retire the second scorer in `app/index.html` . (Now the only failing test in the suite, and the only one at the `v3.31.0` tag.) It assigns **21 of 56 features** — 35 are never written and silently read 0, and 8 of the 21 disagree with the engine outright. This is the "facts are global" rule broken inside the UI. `app/scoreboard.html` already does it correctly by rendering engine-computed values. The real fix is to ship the actual engine to the browser — `medicham2-browser.js` already runs in both, and `board.js` has no top-level requires; the only blocker is one `process.env` read at module scope.

7. Bring the five living docs current, and keep them current. Whitepaper, deck, technical docs, SUMMARY and MODELS drifted to 3.21.0 against a CHANGELOG at 3.28.0 once already — see the process failure recorded in 2.6. SUMMARY, MODELS, the white paper, PUBLICATION and this file were brought to 3.31.0 on 2026-07-31; the deck and technical docs have NOT been checked since.

8. Measure exploitability again, against the mixed-strategy version. The test that says whether the project has moved from *plays okay* onto the branch that can approach *solved*.

9. External validation. Our bots grading each other cannot establish competence. The two real scoreboards are the Bo3 open-sheet ladder — where the hidden-information problem does not exist and we already collect the games — and VGC-Bench's published agent results on that same format.

4. THE SEARCH REDESIGN — established 2026-08-06

This section exists because a conversation about *why the bot picks badly* turned up four things that were true, written down, and unacted on. Every figure below was read out of a file or measured in this session; two of them correct claims I made earlier in the same session from memory.

4.1 The structural fact everything else follows from — and it was already written down

Real games are six turns. Measured over 53,059 stored games:

median	mean	p90	p99	over 30 turns	over 60 turns
6	6.5	10	16	0.05%	0.01%

The rollout horizon is capped at **60**. That is $\sim 10\times$ the real game, and reasoning from the cap rather than the distribution is how the cost model came to be $10\times$ too big: a leaf evaluation is about **5,600** move-decisions, not 48,000.

docs/POKER-TO-POKEMON.md **\$4b had this number and its consequence before tonight:**

*"the horizon is short: the median game is 6 turns... the binding constraint is **breadth, not depth** — action abstraction (pruning dominated moves, bucketing targets) is where the real work is, **more so than the value net.**"*

Two things follow, and the second is the uncomfortable one:

- **A leaf matters less here than in poker or Go.** A leaf evaluator exists because you cannot reach the end of the game. At six turns the rollout *does* reach the end — it returns a real win or loss. There is nothing to approximate. **The problem is not the judge at the bottom of the tree; it is that the six turns leading to it are played by a coin.**
- Which means the PORYGON2 separation gate (#23), dispatched earlier the same evening, is answering a question of smaller consequence than the dispatch brief claimed. It is still worth having. It is not the thing that unblocks anything.

4.2 GARY — the opponent inside the search, now named, and built-and-switched-off

A capability that cannot prove it ran is assumed broken. GARY could not, because it had no name.

```
engine/mag_bot.js:173      const MILTANK_FOE = arg('miltank-foe', 'uniform');
engine/miltank.js:455      DEFAULTS = { ... foePolicy: 'uniform', ... }
engine/rollout_leaf.js:289 const mv = (foePolicy === 'prior' && pickByPrior(mon, rng)) || mvs[random];
```

'prior' samples data/move-priors.json — **128,548 recorded clicks over 295 species** — so the imagined opponent clicks Protect and Fake Out at the rate real players do instead of 25% each. It is wired end to end and **the default is the coin, in the library and in the live bot.**

Four separate defects, each recorded as its own item:

1. foePolicy **steers both sides.** rollout_leaf.js:302-303 applies the same pick to S.actA and S.actB. One flag governs the whole imagined game, so you cannot give the opponent a brain without also changing how the search models itself. (#34)
2. **Targeting is random in both settings.** rollout_leaf.js:290 draws the target uniformly whether or not prior is on. prior fixes *which move* and never *who it hits* — and in doubles the target is half the decision. board.js:377 already records humans double-targeting 23.4% of the time against $\sim 50\%$ for independent choice. (#35)
3. **GARY has two seats and they disagree.** From rolloutAfterActions's own comment: *"The opponent is NOT modelled. It plays chooseAction during the stepped turn."* Deterministic greedy on the turn being ranked; a coin for every turn after. (#36)
4. **No artifact records which GARY ran.** data/rollout-r1.json and data/rollout-r1-explore-sweep.json carry no foePolicy key, so R1 and R4 cannot say whether their opponent was a person or a coin. (#33)

Correction to something asserted earlier this session: MAG is *not* deterministic and the "sampling would collapse the variance" objection was wrong. greedy=false already draws from a softmax, and magnemite.js:217 calls it *"the single biggest measured lever in the project."*

4.3 The pruning is done by the coin

miltank.js:1204 screens every candidate pair by running a **cheap rollout** on it and keeps the top FINAL_K . The rollout is the 51%-accurate leaf. **So the coin is not merely misjudging the finalists — it is selecting them.**

MAG is not used for this, and the file says why:

"The first version pruned to the best K per slot — except _candsFor returns no scores, so it was taking the first three in array order and calling that the best three. That is worse than not pruning at all: an arbitrary shortlist that LOOKS principled."

The candidate list arrives with no scores attached. Fixing that is \$4b's "action abstraction", and it costs *one* MAG pass per turn rather than one per imagined turn — the difference between affordable and impossible. Measured branching, from 7,976 real brought-teams: ~76 action combos per side per turn, ~5,738 joint. MILTANK samples 200 rollouts against that. (#37)

4.4 The equilibrium solver is unplugged, and the reason is the language boundary

How the comparable projects handle the opponent: they do not pick one. CFR → DeepStack → Libratus → ReBeL never assume an opponent policy; they run regret matching for both players jointly and output a *mixed strategy*, whose guarantee is that it cannot be exploited whatever the opponent does. AlphaZero uses self-play with a policy prior and deletes rollouts entirely. AlphaStar — the closest structural analogue, being simultaneous, hidden-information and wide — uses a *league* including deliberate exploiters, which is what WOBBUFFET is here.

The poker answer is already implemented in this repository, verified, and unreachable.

engine/slowking/nash.py and ismcts.py do simultaneous-move regret matching and recover exact Nash on RPS and an asymmetric 2×2. rollout_leaf.js states the gap plainly: *"a best response to a fixed opponent rather than an equilibrium: weaker than the design's matrix game."*

127 JavaScript files	everything that must PLAY a battle — Showdown is TypeScript
40 Python files	everything that does MATH — numpy/scipy live there
7 in slowking/	the solver

Neither language choice was wrong. **The consequence is that the thinking cannot play.** Three routes out, and the third is the one to scope first:

route	cost
port the solver to JS	a second implementation of verified math — the failure this repo keeps repeating
call Python per turn	a subprocess inside a Showdown turn timer
precompute in Python, ship a lookup table	what chess does with Syzygy — and it is DUSK

4.5 DUSK is a tablebase, and open sheets are what make it exact

Will, 2026-08-06: *"at the end game, we can have a repository of scenarios in dusk that can show which mons beat which in a straight up battle and solve for those endings."*

That is an endgame tablebase, and it is the right shape for this game specifically. At 1v1 under **open team sheets there is nothing hidden left** — species, set, item, ability and nature are all declared. The only unknown is which of four moves they pick this turn: a small matrix game with known payoffs, which is exactly what nash.py already solves exactly. **Closed sheets would make the table a guess; open sheets make it a fact**, which is the payoff of the open-sheet-only directive.

The store supports reconstructing these positions — turn records carry `tgthp`, boosts and megas, and `sets` carries `declared:true`.

And DUSK is simultaneously the bridge in 4.4: solve offline in Python, emit a table, let the JavaScript bot read it. No port, no subprocess.

The open question is size, and it decides everything: enumerate all of (mon + set + HP + status + boosts)² × field and it is astronomical; restrict to positions that actually occur on the ladder and it may ship as a JSON file. Not yet measured. (#40)

4.6 A coverage number that overstates itself ~3×

`data/mechanics-census.json` : **probed 219, live 216, missing 3 — armed 74, unarmed 145**. An unarmed probe runs, reports a result, and would report the same result if the mechanic were deleted. So "216/219 mechanics live" counts probes that executed, not mechanics that work.

The instrument that does *not* depend on anyone thinking of the right probe — random games diffed against real Showdown — finds **1 disagreement in 150**, and that is the honest signal. It is good. The remedy is arming the 145, not writing more. (#42)

4.6b Speed is revealed for free by resolution order, and nothing reads it

Will, 2026-08-06: *"if two incins come out, which intimid goes first indicates speed."*

Entry abilities resolve in **speed order**, so two Intimidates on the same switch reveal a strict speed inequality **before anybody clicks a move**. It costs no turn and no risk. The same channel has several sources: any two entry abilities (Drizzle against Drought), end-of-turn residuals (Leftovers, burn, Rough Skin), and same-bracket move order (two Fake Outs, two Protects).

This matters more than it sounds, because of what open sheets leave hidden. `web/models.html` 's own mode note: *"Hidden information is exactly two things: what's in the back and the exact EV spread."* Species, item, ability, nature and moves are all declared. **So Speed is essentially the hidden variable in this format**, and every resolution-order observation is a hard constraint on it. Accumulated across a battle it narrows to an interval, which is exactly the *"do I outspeed this"* question that decided three of the five replays reviewed the same evening.

What exists is not this. `engine/dynamics.js` → `data/dynamics.json` already derives speed from *who moved first* — but it reads **move order only**, aggregates **per species over the whole corpus** into an empirical rank rather than per-battle per-opponent, and is read by `ditto.js` and `kadabra.js` and by **nothing** in `board.js`, `miltank.js`, `mag_bot.js`, `magnemite.js` or `medicham2-browser.js`. It is a population prior, not a live belief.

And entry-ability order is modelled nowhere at all — `entryOrder|switchInOrder|abilityOrder|speedOrder` returns no match across all 127 JavaScript files. `medicham2-browser.js:2444` applies entry drops and carries no notion of which resolved first.

Three pieces, and **the first is an ENGINE correctness question rather than a feature**: does MEDICHAM resolve entry abilities in speed order at all? The official engine does, so if MEDICHAM does not, that is a differential-test miss to fix before anything is built on top of it. Then the extraction (protocol order → a qualified inequality), then the belief (inequalities → a Speed interval a decision can read). ****XATU** is currently drawn as *"mostly idle — little left to guess"* under open sheets. This is the thing it should be doing instead.** (#43)

4.7 The order, and why each is where it is

#	item	why here	cost
#33	record which GARY ran	nothing below is interpretable without it	trivial
#32	flip uniform → prior, measured	free, and it gates whether a better GARY is worth building	a flag
#38	set the horizon from the measured 6	every cost estimate downstream depends on it	trivial
#37	prune with MAG's scores, not the coin	§4b's "the real work"; one MAG pass per turn	small
#34 #35 #36	split the policy, model targeting, unify the seats	make GARY one declared thing	medium
#39	measure the board↔MEDICHAM translation	decides whether MAG-as-GARY is possible at all	measurement
#40	DUSK size gate	decides whether the tablebase is a weekend or a year	measurement
#41	reach the solver from JS, via #40	the equilibrium answer exists and cannot play	scoping
#42	arm the census	stops an inflated number being quoted again	ongoing
#43	speed from resolution order — and Foul Play goes further, running the damage calc BACKWARDS : it simulates rolls against the damage actually observed to infer the opponent's real stats, and reads items from behaviour (outspeeds you repeatedly with the same move → Choice Scarf). We infer nothing from either channel (#74)	free information on the one variable open sheets still hide	ENGINE check, then small

The through-line: four of the six findings in this section are capabilities that were *built, correct, and switched off* — prior, MAG's scores at the candidate list, MAG's softmax, and the Nash solver. That is the same failure mode as 2026-07-28, arriving through a door the rules had not covered, and the countermeasure is the same one: **a capability that cannot prove it ran is assumed broken**. GARY is named in this pass for exactly that reason.

5. THE REGISTER — everything we intend to build, and why it is not built yet

Will, 2026-08-06: "I WANT THE ROADMAP TO BE COMPREHENSIVE AND UNABLE TO LOSE TRACK OF THE THINGS WE WANT TO BUILD LATER."

The rules, because a list that is not checked is a wish-list.

1. An item leaves this register in exactly two ways: **it is done**, or **Will kills it by name**. Nothing leaves by being forgotten, superseded in conversation, or quietly reprioritised.

- Every item names **what unblocks it**. "Later" is not a status; "blocked by #45" is.
- Every item carries a **task number**, and the task holds the detail. The register is the index — it exists so that nothing is only in someone's head, or only in a conversation, or only in a division ledger nobody opens.
- A deferred item stays here with its reason**. HYPNO is deferred by decision, not dropped, and it says so.

5.0 THE FOUR PHASES — the ordering this register is arranged in (3.59.0, ADR-003)

This section is new and it re-orders everything under it. It exists because ADR-003 changed what the project is trying to prove. The headline metric is **exploitability, not win rate**; VGC-Bench (AAMAS 2026) beat a professional with a compiled policy and measured all of its agents at approximately **100% exploitable**; and the thesis under test is that a **re-solving** agent is harder to exploit than a compiled one. That thesis makes MEDICHAM's existence conditional, and the condition is a gate that was already in this register.

```
1  finish MEDICHAM          search needs an engine that is fast AND correct
2  GATE #62                 does compute buy anything: untimed vs on-the-clock
3  if yes -> search, and measure EXPLOITABILITY against their ~100%
4  if no -> adopt their recipe: BC + PPO self-play/FP/DO, open source, reproducible
```

phase	what it is	where it lives in this register
1	MEDICHAM is fast and correct	§5.1, and #68 as the whole-engine gate
2	#62 — the project gate , below	this section
3	the search, then the exploitability measurement	§5.2, §5.3, §5.4, and #6 in §5.3
4	BC + PPO with self-play / fictitious play / double oracle	§5.8, as the paper's fallback result

#62 IS PROMOTED TO THE FRONT OF THIS REGISTER. It is no longer a MILTANK tuning question; it is the project's gate, and it was moved here from §5.3 in this pass.

#	item	blocked by
#62	GATE, AND IT COMES FIRST: play MILTANK untimed against MILTANK on the clock. R6 measured 31.6% of move decisions deferred — handed back to MAG unsearched, consuming 30.5% of the spend. defer fires both when the clock runs out <i>and</i> when the search cannot separate its options, and only the first is fixable with compute . This decides whether #25/#37/#61 are worth doing at all — and, since 3.59.0, whether the whole engine programme was worth doing. VGC-Bench used real Showdown through poke-env and carried no engine-correctness debt , because behaviour cloning and PPO do not need a fast simulator. We wrote one so per-turn re-solving is affordable, so the engine work is justified if and only if search pays	#45

Why this is the right gate rather than a proxy for one. Every project in docs/PRIOR-ART.md that searches hits the engine-speed wall, and the pattern is clean: VGC-Bench does not search and used real Showdown; Future Sight AI searches, modified Showdown, and reaches about 3 turns of lookahead in 15 seconds on 16 cores; Foul Play searches, **built** poke-engine, and reaches about 10+ turns. Building an engine is what searching costs. #62 asks whether the thing it buys exists.

On compute, because it is the obvious response to #62 failing. Cores help the search — it is CPU-bound and root-parallelisable. GPUs help behaviour cloning and PPO. MILTANK needs **26 s against a 20 s budget on one core of sixteen**, so sixteen cores fixes the clock today. But root parallelisation scales **sublinearly**, so cores convert a *failed budget into a met one*, not a *shallow search into a deep one*. If #62 says compute buys nothing at depth, more cores does not answer it.

Phase 4 is approved in advance and is a result, not a defeat. Will, 2026-08-06: *"IM OKAY TRYING THINGS OUT LIKE SEARCH TO SEE IF OUR ATTEMPT WORKS BUT IF THAT FAILS IM OKAY ALSO TRYING THEIR OTHER METHODS."* Their recipe is published, open-source and reproducible, so the fallback is cheap precisely because they made it so.

5.1 MEDICHAM completeness — PHASE 1, and everything below waits on it

Will's bar, 2026-08-06: *"i still want medicham to be fully wired and tested on every move and ability and item in the regulation (with any usage at all) before we start taking its output and using them."* Target agreed the same day: **99% of usage — 484 of the 819 things that carry real usage** — plus a carve-out for anything that turns a certainty into a failure regardless of usage.

#	item	status
#45	the gate itself: 99%-of-usage coverage, enforced not remembered	being built
#42	ratchet on DIRECT-CALL probes, not on <code>unarmed</code> — 37 left of 47	in progress
#51	23 tags with real usage and no probe at all ; accuracy modification is the top	queued behind 42
—	3 declared-missing mechanics: Avalanche, No Guard, Sand Veil	queued behind 42
—	1 live differential: <code>chesnaught woodhammer -> mimikyu</code> (Disguise, unimplemented)	queued behind 42
—	6 interaction-matrix disagreements: stone axe ×4, giga impact, supercell slam	queued behind 42
#27	the represented-clicks number — what % of real clicks the engine can even perform. Last measured 15.3% failing when it handled 4 action kinds; it now handles 12	needs a settled engine + fresh release
#49	delete the 77 hand-typed mega abilities; the derived path exists and is proven equal	queued behind 42
#9	harden <code>pranksterBlocked</code> to check the move's target	latent, not live
#20	a TYPE cannot be a reactor, so Grass-blocks-powder is untestable	—
#21	AXIS 4 — DENIAL: a move that costs a turn also denies a field effect	—
#28	resist berries: the one staged pair is physically impossible	—
#7	Coverage Layer 2 — the mutation harness	—
#59	the protection counter has three behaviours and our tags carry two. Protect <i>checks</i> the shared 1/X counter; Wide Guard <i>feeds it and never checks</i> ; Ally Switch has its own private one and carries neither tag. The engine implements none of it — <code>grep allyswitch</code> and <code>grep stalling</code> both come back empty	—
#60	Upper Hand and Sucker Punch share a tag and have different conditions — Sucker Punch needs the target attacking, Upper Hand needs it attacking <i>with priority</i> . Modelled the broad way, the bot thinks Upper Hand beats an ordinary Earthquake	—

Both came from Will reading the rules, not from any instrument, and both are the same shape: a tag derived from Showdown's flags is only as complete as the flag. Ally Switch's stalling lives in `onPrepareHit` with no flag on it, so the deriver never saw it. **Showdown does not have tags — it has code**, and every mechanic here is a re-implementation of a handler, which is the entire reason `engine/tag_dex.js`, the differential test and the arming work all exist.

5.2 GARY — the opponent inside the search, named 2026-08-06

Four defects in one place, all found by reading the source. See §4.2.

#	item	blocked by
#33	every artifact records which GARY ran — <code>rollout_r1.js</code> done, 3 callers left	—

#	item	blocked by
#32	flip <code>uniform</code> → <code>prior</code> and measure it	33, and a fresh release
#34	the flag steers my side too — split it	—
#35	targeting is drawn uniformly in both modes. Foul Play weights its chance nodes by likelihood — a 90/10 accuracy move becomes two children selected by probability. It does correctly what #32/#35 have open as a defect (#74)	—
#36	GARY has two seats and they disagree	—
#55	GARY does not coordinate either — the imagined opponent picks its two Pokémon independently, so it never focus-fires, never Protects while its partner kills, never redirects. **DODUO is GARY** , pointed at the other side	#39, #45

The fifth defect is the largest and it collapses two workstreams into one. MAG choosing independently costs us our own quality — a symmetric loss. **GARY choosing independently makes the imagined opponent structurally incapable of the plays that actually beat us**, which is bias with a direction. And the objection that imitation is a ceiling *does not apply to the opponent model*: we **want** the imagined foe to behave like a human, because humans are who we play.

5.3 The search redesign — PHASE 3, and none of it starts until #62 answers

Reordered 3.59.0. This whole table used to be the front of the project. It is now downstream of the gate in §5.0: if compute buys nothing, none of these items is worth building, and the project takes phase 4 instead. **#61 keeps its place here and gains a second reading** — the 3,401 battles/sec it is measured against is itself corrected in 3.59.0 to 13,041 turns/sec against `champions_sim`'s 523, a ratio of 24.9x rather than the recorded 117x. Three readings of one quantity, no ratchet on any of them; see ADR-001's correction note.

#	item	note
#37	prune with MAG's scores, not with the coin	Will approved with a condition: <i>"make sure it doesnt toss moves a VGC pro would make"</i>
#25	prune by PAIR score, not single-move	the truncation curve says single-move ranking cannot meet that condition at any affordable K
#53	DODUO — the pair model is fitted, better, and the search never reads it	the fix is wiring, not building
#56	extract what MAG actually prunes that DODUO wants — the truncation misses <i>are</i> the catalogue, and reading them out beats reasoning from examples	needs a frozen release
#24	replace MILTANK's leaf with PORYGON2 + a MAG-sampled opponent, and add Foul Play's damage-roll grouping as a third arm — group outcomes by whether they cause a FAINT, average the damage inside each group, sum the likelihoods. Keeps the only distinction the decision turns on and collapses the rest; cheaper AND lower-variance without being dumber, which is the exact pair of properties this task is short of (#74)	gate passed (#23)
#38	the rollout cap is 60 turns; real games end at 6	trivial
#39	measure the <code>board.js</code> ↔ MEDICHAM translation cost	decides whether MAG-as-GARY is possible at all
#61	MEDICHAM measures 1,606 battles/sec against the 3,401 on record — 47%. On the measured figure the 200-rollout x 64-pair search needs 26 s against a 20 s budget and does not fit. Nothing ratchets speed, which is why a 2x regression went unseen	—
#62	MOVED TO §5.0 IN 3.59.0 — it is the project gate, not a search-redesign item. Everything in this table is phase 3 and does not start until #62 answers	see §5.0

#	item	note
#63	the rollout never switches. The engine <i>can</i> , and the search can offer a switch as a root candidate — but the playout has no switch branch at all, so every imagined future is a game where nobody ever leaves . Misprices preserving a Pokémon, punishing a switch, and the <i>switch out of a predicted attack</i> pattern the engine models priority order for	#45, #61

Four different states get quoted for each other here and only one is broken. The engine can switch; the search can offer it; **the rollout never does**; and the live bot has it **off** against a measured **10-point loss**. That loss is real — but `bringIn()` picks `live(bench)[0]`, the first healthy body, and its own comment calls that *"a real limitation, not a detail"*. ****A switch to an arbitrary Pokémon is not a switch, it is a leave**** — and a 10-point loss is precisely what evaluating "leave" would produce. Test that before concluding switching is bad.

5.4 Solving — DUSK, and the retrograde idea

#	item	note
#47	solve exactly whenever switching is off the table — 1v1 is 16 cells, 2v1 is 96, 2v2 is ~1,296	2v1 alone reaches 45.9% of games
#47	retrograde analysis (Will): with 2v2 solved, a 3v2 reduces to <i>which solved 2v2 do I steer into</i>	this is how chess tablebases are built
#29	THE SACK — a planned sacrifice is invisible to a material-weighted leaf	it is the same idea: the sacrifice is the move that reaches a won position
#41	reach the Python equilibrium solver from the JavaScript bot	a small-case port is verifiable against <code>nash.py</code> ; a full port is not
#30	DUSK as a goal — nothing in the project represents a goal, only positions and actions	

5.5 Built, measured, and never used in a live decision

The recurring failure of this project, kept together so the pattern stays visible.

model	state
DODUO	fitted, refitted, +2.1 top-1 over independent choice, never read by the search (#53)
PORYGON2	built, separation gate passed, never called by a live decision (#24)
GARY prior	wired end to end, default is a coin (#32)
MAG's scores at the candidate list	not attached, so the screen prunes with the rollout instead (#37)
MACHAMP	<i>"half-run and stale — the single largest untested lever in the project"</i> ; artifact deleted 2026-08-02
the Nash solver	verified on RPS and a 2×2, in Python, unreachable from the bot (#41)

5.6 Measurement integrity

#	item
#26	refit MAG on bo3 only — the corpus change is in code and unrun
#50	WIRE 124 moved a position feature; models fitted on it predate the correct number
#48	27 of 28 Python generators could write a file JavaScript cannot read — closed, gated
#27	three instruments built that had never written an artifact — truncation curve now run , two left
#44	a move that provably does nothing is a weighted preference, not a rule — needs a refit
#57	every rollout-derived number predates a correct engine — the leaf's 51.0%, R1's 69.84%, R4's 55.5%. Enumerate the re-run list from the release stamps; prioritise by whether the verdict could flip, not by age
#58	two <code>moveAccuracy</code> implementations disagreed for 78 moves and nothing compared them. <code>board.js</code> read the dex and was right; <code>medicham2</code> read a hand-typed list and was wrong. Make it one function and assert it

What is safe and what is not, because the distinction decides what has to be re-run. Anything that ranks or predicts through `board.js` never touched the broken simulator — the truncation curve, DODUO's top-1 figures, MAG's realism table. Anything that runs a **rollout** went through `medicham2`, where until 2026-08-06 the weather resolved backwards from turn one, 78 moves could not miss, Last Respects reverted, and a type-converted move dealt zero. **That is not a claim the numbers were wrong** — WIRE 123's error was symmetric across both arms of a paired comparison, so a contrast may survive it while an absolute rate does not. Establishing which is the work.

5.6b Are there too many variables? — audited, and the answer is not the count

76 weights on 81,515 joint turns is ~1,000 observations per parameter, and the fit is stable: handed 15,279 further turns it flipped **none** of them. The problem is not size.

`data/collinearity-joint.json` found the real shape: **8 of the 18 coordination features fire on under 1% of decisions** — `boostsPartnerDamage`, `boostMayConvertKill`, `weatherSetupHelpsPartner`, `healsPartner`, `redirectThenSetup`, `doubleKO`, `terrainSetupHelpsPartner`, `screenWhileThreatened` — and three of the live ones are correlated enough that their individual weights are not interpretable (`bothSameTarget` ↔ `overkill` $r=0.53$, `bothSameTarget` ↔ `focusFireKills` $r=0.47$).

Its own honest caveat, which must travel with the finding: *"Collinear coefficients can be individually meaningless while the MODEL is well calibrated, because the correlated block still carries the right total."* So this is not evidence the pair model predicts badly. It is evidence about which single weights can be read, and about a third of the block that rarely does anything.

5.6c Self-play cannot find a shared blind spot

Kept in the register because it is the standing reason not to trust a head-to-head on its own. CLAUDE.md: *"a head-to-head, an exploitability search and a prediction score all compare two bots that **share the blind spot**, so a missing capability **cancels out exactly**."*

Every bug found on 2026-08-06 would be invisible in self-play — both sides had the weather backwards, both believed the same 78 moves could not miss. And it is not theoretical: self-play training measured **48.1% [46.5, 49.8]** over 9,728 paired games, an interval entirely below 50. The guard that exists is MACHAMP's — every promoted champion plays **every** previous generation, not only the one it displaced, which is the only thing that would detect a cycle.

5.7 Deferred by decision — not dropped

#	item	why it is here
#52	HYPNO — rates the opponent, sets how hard to exploit	Will, 2026-08-06: <i>"NAH LETS HOLD OFF ON HYPNO BUT I DONT WANT TO FORGET ABOUT IT."</i> Worth remembering that it is the only model on the board whose inputs do not include MEDICHAM , so it is never blocked by §5.1 and never needs re-running when the engine changes.

5.8 The paper

#	item
#54	position against THREE comparables, because the field is not empty and none of them is playing our game. VGC-Bench (AAMAS 2026) — whose archive we already consume, and dropped from the fit for staleness (four days, mid-June, seven weeks old); <code>engine/corpus_shift.js</code> exists to measure the metagame distance and has never written an artifact. Future Sight AI — closed-source Showdown bot, reportedly top 5% of the most popular format, matches revealed attributes to the most common set, ships a win-probability browser extension; also settles a naming question, since "Future Sight" is TAKEN and this project names everything after Pokémon. The PokeAgent Challenge (arXiv 2603.15563, NeurIPS 2025, 100+ teams, two tracks) — by far the strongest reference of the three and not yet read . Add Sarantinos (arXiv 2212.13338, Cambridge) as the closest published method: Gen 7 Random Battles singles, peaked 33rd in the world , regret-minimisation opponent prediction, fills unknown properties from usage statistics, and explicitly rejects Monte

#	item
	Carlo sampling as too sample-inefficient — which is what MILTANK's leaf does. It also names our #29 independently: its first "biggest challenge" is keeping the team balanced, with a worked example that is THE SACK, and it asserts <i>"an AI agent cannot achieve this just by looking ahead with MinMax or Monte Carlo Tree Search."</i> Finally, its ELO caveat is load-bearing for Will's stated goal: comparing human and AI ratings is invalid unless they played each other — laddering with ALAKAZAM is exactly that, so the plan is sound and should say why
—	the candidate contributions are the truncation curve , the represented-clicks curve , the armed-coverage critique , and the negative results — a separation gate that uniform noise passes, a tablebase killed by reach rate, and per-turn metrics that cannot separate a 1043 player from a 1600
—	PHASE 4, and it is a paper either way (3.59.0) . If §5.0's gate says compute buys nothing, the project adopts VGC-Bench's recipe — behaviour cloning plus PPO under self-play, fictitious play and double oracle — and reports that as the result. Will approved this branch in advance. Either way the instrumentation is the honest contribution : no project in docs/PRIOR-ART.md publishes a mechanics census that must be shown RED before it counts, a step-level protocol differential against the official engine, ratchets on silent failure, or a record of what it retracted. If search loses, this is still the only account of what a hand-written VGC simulator gets wrong and how you would know

The honest summary

What is genuinely strong: the data discipline, the validated engine, the measurement culture, and the tooling that now enforces all three. The project's most valuable output to date is not a model — it is the ability to tell which of its own results are real, which was demonstrated by dissolving four of them in a single day.

What is not: every in-battle model. MAG is a competent starting position and a measurably exploitable one. No model in this project has yet been shown to play VGC well against anything other than a worse version of itself.

The gap between those two paragraphs is the whole roadmap.